UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



REDES DE COMPUTADORAS

Trabajo práctico N°5

Docente: Santiago Martin Henn
Nombre de Grupo: Death Net
Integrantes:

- Alvarez Quiñones, Andrés
- Amarante, Flavio Eduardo
- Borchi, Pehuen Emmanuel
- Carreras, Leonor Victoria
- Hernandez Monagas, Estefany

-2026-

DESARROLLO:

a) ¿Qué es ICMP y para qué se usa? ICMP (Internet Control Message Protocol o Protocolo de Mensajes de Control de Internet) es un protocolo auxiliar e indispensable que forma parte de la Capa de Red (IP).

Su función principal es permitir que los routers y los hosts (equipos finales) intercambien información de control, diagnóstico y notificación de errores que ocurren durante la transmisión de paquetes en la interred.

Principales usos de ICMP:

1. Notificación de errores:Cuando un paquete IP no puede ser entregado o procesado correctamente, un router o el host de destino genera un mensaje ICMP dirigido al emisor original especificando la causa del problema:
    - **Destino inalcanzable (Destination Unreachable):** La red, host, puerto o protocolo de destino no está accesible.En IPv4, si un datagrama requiere fragmentación pero tiene activo el bit DF (Don't Fragment), se notifica con este mismo mensaje (Código 4: Fragmentation Needed). En IPv6, esta situación se notifica mediante un mensaje específico denominado Paquete demasiado grande (Packet Too Big).
    - **Tiempo de vida expirado (Time Exceeded):** Ocurre cuando el campo TTL (IPv4) o Hop Limit (IPv6) llega a 0 antes de alcanzar el destino, evitando que el paquete circule indefinidamente en un bucle.
    - **Aclaración:** ICMP notifica determinados errores, pero no todas las pérdidas de paquetes generan un mensaje ICMP. Por ejemplo, un datagrama con la cabecera corrupta se descarta en silencio; tampoco se generan mensajes ICMP en respuesta a fallos de otros mensajes ICMP de error (para evitar bucles infinitos) ni para datagramas de multidifusión (multicast).
2. Diagnóstico y pruebas de conectividad:
    - **Ping:** Utiliza los mensajes ICMP de Solicitud de eco (Echo Request) y Respuesta de eco (Echo Reply) para probar si un equipo de destino está activo y medir el tiempo de respuesta (RTT).
    - **Traceroute:** Emplea los mensajes ICMP de expiración de tiempo (TTL) para identificar y rastrear cada uno de los routers intermedios por los que pasa un paquete hasta llegar a su destino.

¿Transporta datos de aplicaciones como lo hacen TCP o UDP?

ICMP no transporta datos de aplicaciones de usuario (como páginas web de HTTP, correos electrónicos de SMTP o archivos de FTP).

Diferencias clave con TCP y UDP:

- Propósito y destino:
    - TCP y UDP operan en la Capa de Transporte y su objetivo es la comunicación proceso a proceso (entre aplicaciones finales), dirigiendo los datos a sockets identificados por números de puerto.

- ICMP opera a nivel de comunicación del sistema/red. Los mensajes ICMP no van dirigidos a una aplicación o programa de usuario, sino al módulo del protocolo IP/ICMP del propio sistema operativo en la máquina receptora.

- Encapsulamiento:
  - Aunque ICMP se ubica en la capa de red, sus mensajes se encapsulan directamente en la zona de carga útil de un datagrama IP (con el valor de protocolo '**1**' en la cabecera IP). A pesar de ser transportado por IP, sirve para dar soporte y reportar fallos de la propia infraestructura de red, no para transportar el flujo de información generado por los usuarios.

b) ¿Qué relación tiene con IP?

Es un componente obligatorio e integral de la Capa de Red (Capa 3): ICMP forma parte del propio estándar del protocolo IP y debe estar incluido en todas las implementaciones de la pila de protocolos TCP/IP.

Mecanismo de soporte y notificación: No existe como un protocolo independiente ni de aplicación, sino que fue creado específicamente para servir de herramienta de control y reporte de errores al protocolo IP. Permite que los routers y hosts señalen situaciones anómalas (como la pérdida de un datagrama, congestión o fallos de ruta) a la máquina de origen.

¿Viaja dentro de IP, al lado de IP o debajo de IP?

Viaja dentro de IP.Todo mensaje ICMP se transporta encapsulado en la zona de carga útil (payload) de un datagrama IP.

Doble nivel de encapsulamiento: A nivel de trama de red, el mensaje viaja ordenado de la siguiente manera :

[Cabecera de Enlace (Ethernet)] -> [Cabecera IP |Cabecera ICMP|Datos ICMP]

Estatus arquitectónico: Aunque viaja dentro de un datagrama IP, ICMP no se considera un protocolo de capa superior (de transporte), sino un protocolo de Capa 3 que aprovecha la misma infraestructura de reenvío de IP para poder atravesar múltiples redes hasta llegar al origen.

¿Cómo sabe el receptor que el contenido de un paquete IP es ICMP?

El sistema operativo receptor identifica la presencia de ICMP consultando un campo específico en la cabecera del datagrama IP:

- **En IPv4:** Lee el campo Protocolo (Protocol) de 8 bits. Cuando este campo tiene el valor entero '**1**', la capa IP reconoce que la carga útil no es un segmento de transporte comercial y la demultiplexa internamente hacia el módulo del protocolo ICMP.

- **En IPv6:** Lee el campo **Siguiente Cabecera** (Next Header), donde el valor reservado para identificar a ICMPv6 es '**58**'.

c) ¿Qué hace ping?

"**ping**" es la herramienta de diagnóstico de redes más extendida para probar la alcanzabilidad y el estado de un equipo de destino en una red IP.

Cuando ejecutas `ping`:

- Envía una serie de mensajes ICMP Echo Request hacia la dirección IP de destino.
- Espera la llegada de los mensajes ICMP Echo Reply correspondientes desde esa dirección.
- Mide el tiempo de ida y vuelta (RTT - Round-Trip Time) en milisegundos y calcula la tasa de pérdida de paquetes, lo que permite verificar si las capas de red, el enrutamiento y los controladores de la máquina remota funcionan correctamente.

¿Qué son un Echo Request y un Echo Reply?

Echo Request (Solicitud de eco): Es el mensaje ICMP de consulta que la máquina de origen envía hacia un equipo remoto para verificar su alcanzabilidad. Puede incluir una sección de datos opcional (por ejemplo, una secuencia fija de bytes para probar la transmisión, la MTU o calcular tiempos de propagación).

Echo Reply (Respuesta de eco): Es el mensaje ICMP devuelto inmediatamente por la máquina de destino al emisor original en respuesta a una solicitud de eco. Este mensaje debe incluir una copia exacta de la carga útil de datos recibida en la solicitud.

¿Qué campos de ICMP permiten distinguirlos?

Todos los mensajes ICMP comparten una cabecera inicial común compuesta por los campos Tipo (Type), Código (Code) y Suma de Comprobación (Checksum).El campo principal que distingue una solicitud de una respuesta es el Tipo:

A) Campo Tipo(Type - 8 bits): Es el campo diferenciador clave.El valor almacenado indica qué formato y significado tiene el mensaje:

En IPv4:

- Tipo = 8 : Identifica un Echo Request.
- Tipo = 0 : Identifica un Echo Reply.

En IPv6:

- Tipo = 128 : Identifica un Echo Request.
- Tipo = 129 : Identifica un Echo Reply.

B) Campo Código(Code - 8 bits): En ambos casos (tanto para la solicitud como para la respuesta), el campo **'Código'** se establece en **'0'.**

C) Campos de coincidencia (Identifier y Sequence Number - 16 bits cada uno): Aunque no diferencian entre solicitud y respuesta por sí solos, el Identificador y el Número de Secuencia se utilizan de manera conjunta para que la máquina emisora pueda relacionar unívocamente cada respuesta con su solicitud:

- Identificador: Permite identificar la sesión o el proceso de 'ping' específico en la máquina emisora (útil si hay varias sesiones de 'ping' ejecutándose en paralelo).
- Número de secuencia: Identifica de forma correlativa a cada paquete individual enviado dentro de esa sesión.
- Uso conjunto: El emisor envía el par (Identificador, Número de secuencia), el receptor lo copia idénticamente en el Echo Reply, y la máquina de origen utiliza ambos valores combinados para asociar la respuesta con su solicitud exacta, medir el RTT individual y detectar paquetes desordenados o perdidos.

d) ¿Qué información mínima contiene un mensaje ICMP de tipo Echo?

Un mensaje ICMP de tipo Echo (tanto para la solicitud Echo Request como para la respuesta Echo Reply) tiene una estructura fija obligatoria de 8 bytes (64 bits) en su cabecera. La información mínima indispensable que componen esos 8 bytes de cabecera es la siguiente:

1) Tipo (Type— 8 bits): Especifica la clase de mensaje Echo.
    - **En IPv4:** valor 8 para Echo Request y 0 para Echo Reply.
    - **En IPv6:** valor 128 para Echo Request y 129 para Echo Reply.
2) **Código (Code— 8 bits):** En los mensajes de tipo Echo, este campo siempre tiene el valor 0.
3) **Suma de comprobación (Checksum— 16 bits):** Campo para la detección de errores calculado sobre la totalidad del mensaje ICMP (cabecera y datos) para garantizar que la información no sufra alteraciones en tránsito.
4) **Identificador (Identifier— 16 bits):** Valor asignado por la máquina emisora (frecuentemente el ID del proceso de la aplicación, como `ping`) para distinguir qué programa o sesión generó la solicitud.
5) **Número de secuencia (Sequence Number— 16 bits):** Contador de 16 bits que se incrementa con cada paquete enviado en una misma sesión para poder emparejar unívocamente cada Echo Reply con su Echo Request correspondiente.

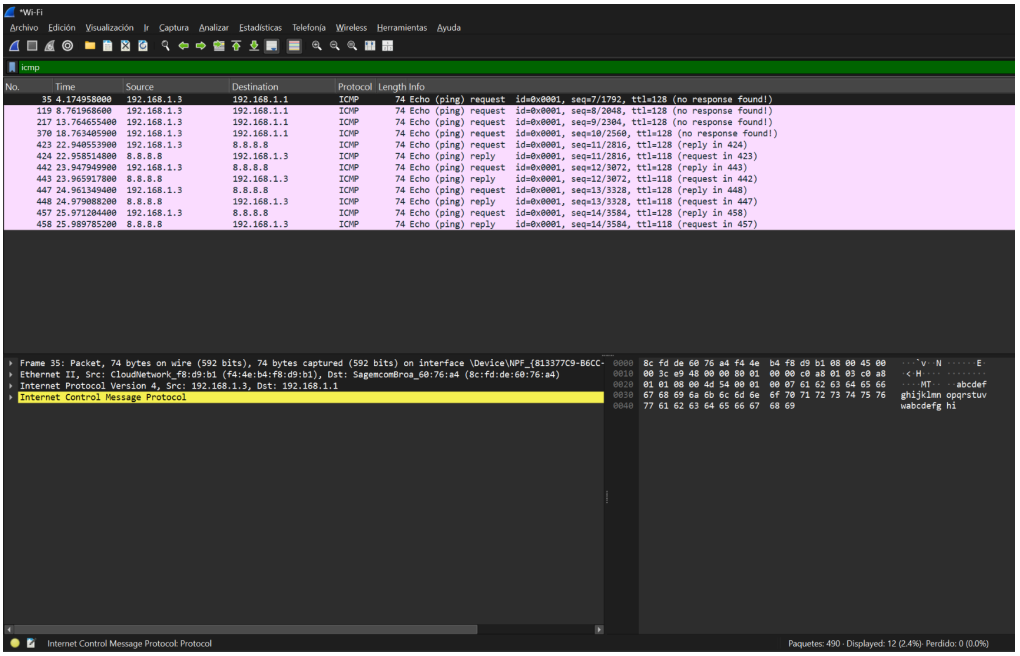
Carga útil de datos (Optional Data):A partir del octavo byte, el mensaje puede incluir un área de datos opcionales de longitud variable (por ejemplo, una secuencia fija de bytes para probar la MTU o marcas de tiempo para medir retardos). La norma ICMP exige que el receptor copie exactamente la misma secuencia de datos recibida en el Echo Request dentro del Echo Reply que envía de vuelta.

Averiguar la configuración de red de alguna de las computadoras del grupo: dirección IPv4, máscara,gateway por defecto y dirección MAC de la interfaz que usan (Wi-Fi o cableada).

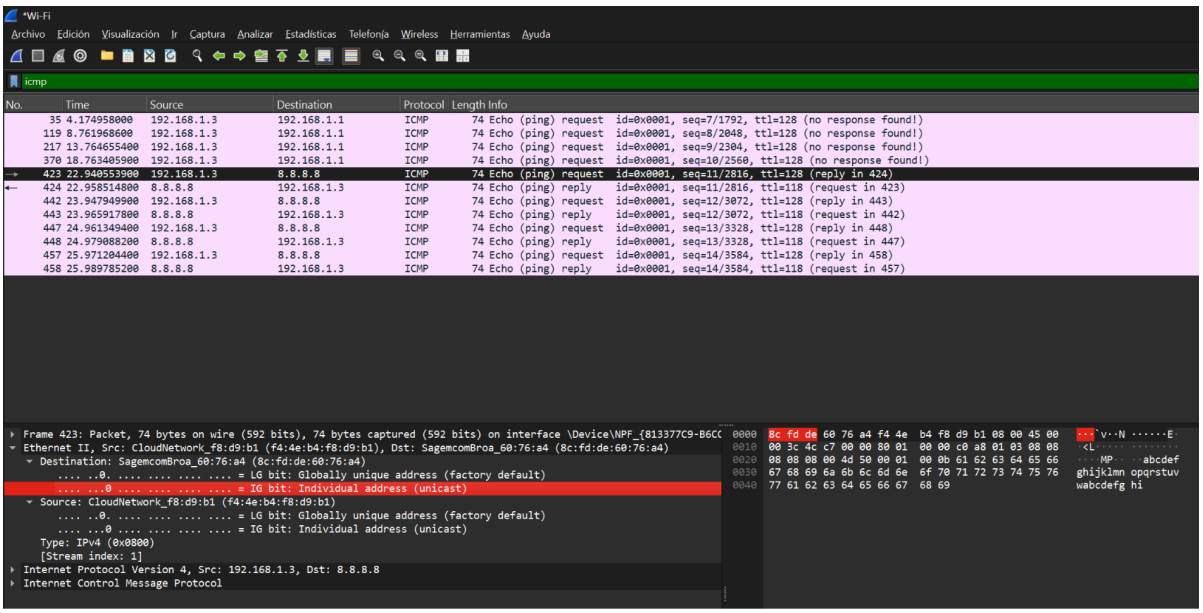
```
Adaptador de LAN inalámbrica Wi-Fi:

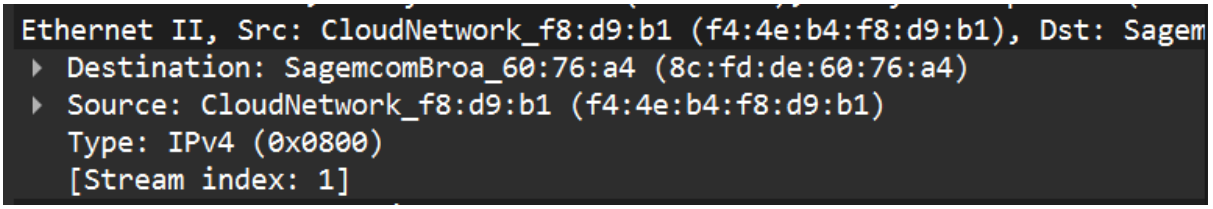
   Sufijo DNS específico para la conexión. . : home
   Descripción . . . . . . . . . . . . . . . : MediaTek Wi-Fi 6 MT7920 Wireless LAN Card
   Dirección física. . . . . . . . . . . . . : F4-4E-B4-F8-D9-B1
   DHCP habilitado . . . . . . . . . . . . . : sí
   Configuración automática habilitada . . . : sí
   Vínculo: dirección IPv6 local. . . : fe80::3c3b:3919:459f:1fd8%10(Preferido)
   Dirección IPv4. . . . . . . . . . . . . . : 192.168.1.3(Preferido)
   Máscara de subred . . . . . . . . . . . . : 255.255.255.0
   Concesión obtenida. . . . . . . . . . . . : miércoles, 30 de septiembre de 2026 10:11:51
   La concesión expira . . . . . . . . . . . : jueves, 1 de octubre de 2026 17:25:02
   Puerta de enlace predeterminada . . . . . : 192.168.1.1
   Servidor DHCP . . . . . . . . . . . . . . : 192.168.1.1
   IAID DHCPv6 . . . . . . . . . . . . . . . : 116674228
   DUID de cliente DHCPv6. . . . . . . . . . : 00-01-00-01-30-55-AF-70-38-A7-46-3D-80-7C
   Servidores DNS. . . . . . . . . . . . . . : 192.168.1.1
   NetBIOS sobre TCP/IP. . . . . . . . . . . : habilitado
```

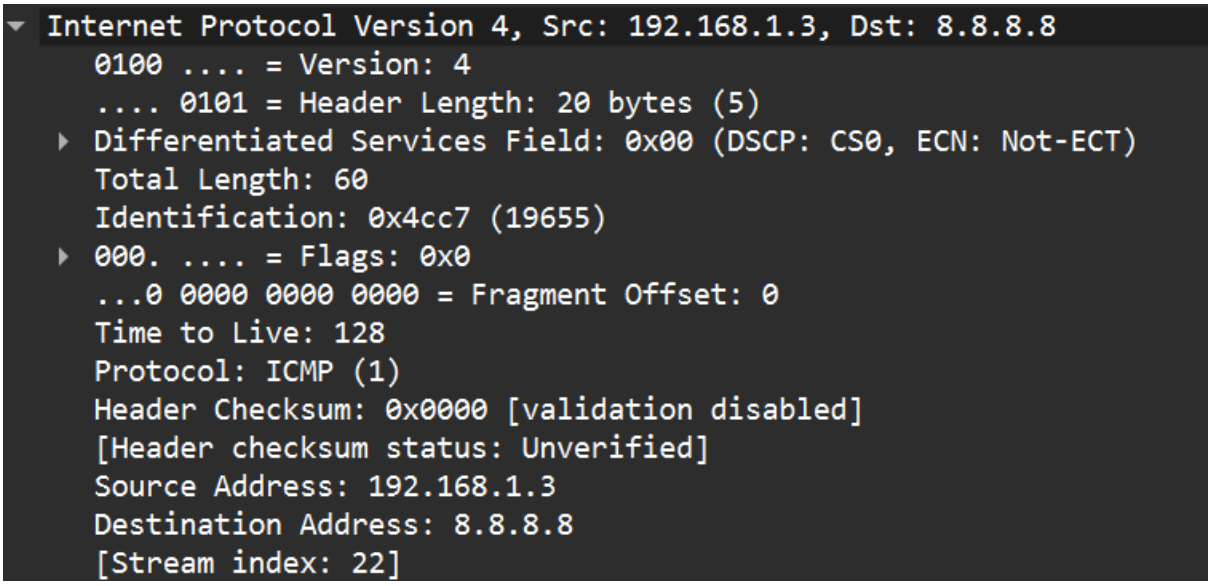
Dirección IPv4. . . . . : 192.168.1.3(Preferido)
Máscara de subred . . . . : 255.255.255.0
Puerta de enlace predeterminada . . . . . : 192.168.1.1
Dirección física. . . . . : F4-4E-B4-F8-D9-B1

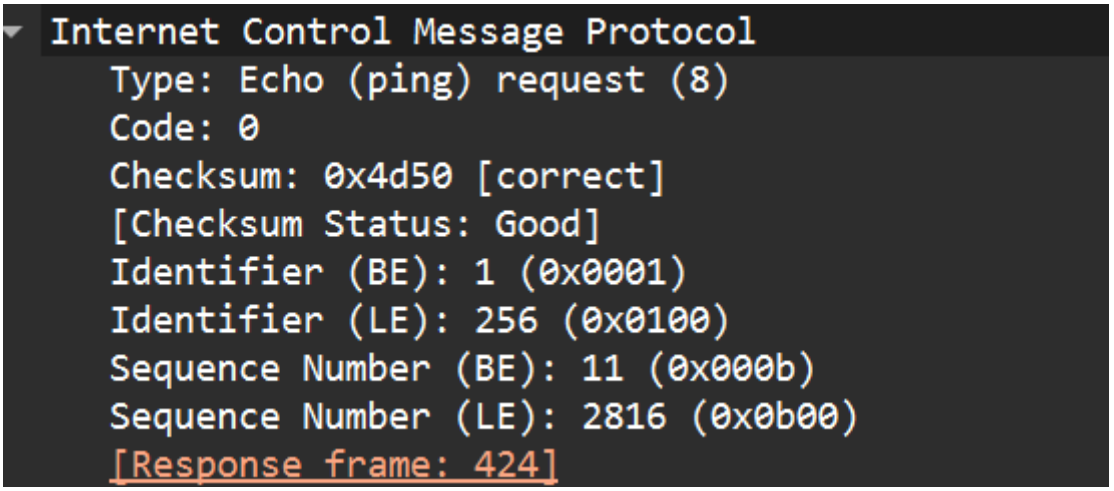


Nos paramos en un paquete Echo Request enviado a Internet ->por ejemplo, el paquete No. 423, que va hacia 8.8.8.8.


*Wi-Fi
Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda
icmp
No. Time Source Destination Protocol Length Info
35 4.174958000 192.168.1.3 192.168.1.1 ICMP 74 Echo (ping) request id=0x0001, seq=7/1792, ttl=128 (no response found!)
119 8.761968600 192.168.1.3 192.168.1.1 ICMP 74 Echo (ping) request id=0x0001, seq=8/2048, ttl=128 (no response found!)
217 13.764655400 192.168.1.3 192.168.1.1 ICMP 74 Echo (ping) request id=0x0001, seq=9/2304, ttl=128 (no response found!)
370 18.763405900 192.168.1.3 192.168.1.1 ICMP 74 Echo (ping) request id=0x0001, seq=10/2560, ttl=128 (no response found!)
423 22.948553900 192.168.1.3 8.8.8.8 ICMP 74 Echo (ping) request id=0x0001, seq=11/2816, ttl=128 (reply in 424)
424 22.958514800 8.8.8.8 192.168.1.3 ICMP 74 Echo (ping) reply id=0x0001, seq=11/2816, ttl=118 (request in 423)
442 23.947949900 192.168.1.3 8.8.8.8 ICMP 74 Echo (ping) request id=0x0001, seq=12/3072, ttl=128 (reply in 443)
443 23.965917800 8.8.8.8 192.168.1.3 ICMP 74 Echo (ping) reply id=0x0001, seq=12/3072, ttl=118 (request in 442)
447 24.961349400 192.168.1.3 8.8.8.8 ICMP 74 Echo (ping) request id=0x0001, seq=13/3328, ttl=128 (reply in 448)
448 24.979088200 8.8.8.8 192.168.1.3 ICMP 74 Echo (ping) reply id=0x0001, seq=13/3328, ttl=118 (request in 447)
457 25.971204400 192.168.1.3 8.8.8.8 ICMP 74 Echo (ping) request id=0x0001, seq=14/3584, ttl=128 (reply in 458)
458 25.989785200 8.8.8.8 192.168.1.3 ICMP 74 Echo (ping) reply id=0x0001, seq=14/3584, ttl=118 (request in 457)
Frame 423: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{813377C9-B6CC
Ethernet II, Src: CloudNetwork_f8:d9:b1 (f4:4e:b4:f8:d9:b1), Dst: SagemcomBroa_60:76:a4 (8c:fd:de:60:76:a4)
Destination: SagemcomBroa_60:76:a4 (8c:fd:de:60:76:a4)
.... ..0. .... = LG bit: Globally unique address (factory default)
....0 .... = IG bit: Individual address (unicast)
Source: CloudNetwork_f8:d9:b1 (f4:4e:b4:f8:d9:b1)
.... ..0. .... = LG bit: Globally unique address (factory default)
....0 .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 1]
Internet Protocol Version 4, Src: 192.168.1.3, Dst: 8.8.8.8
Internet Control Message Protocol
0000 8c fd de 60 76 a4 f4 4e b4 f8 d9 b1 08 00 45 00
0010 00 3c 4c c7 00 00 80 01 00 00 c0 a8 01 03 08 08
0020 08 08 08 00 4d 50 00 01 00 0b 61 62 63 64 65 66
0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76
0040 77 61 62 63 64 65 66 67 68 69


Ethernet II, Src: CloudNetwork_f8:d9:b1 (f4:4e:b4:f8:d9:b1), Dst: Sagem
Destination: SagemcomBroa_60:76:a4 (8c:fd:de:60:76:a4)
Source: CloudNetwork_f8:d9:b1 (f4:4e:b4:f8:d9:b1)
Type: IPv4 (0x0800)
[Stream index: 1]


Internet Protocol Version 4, Src: 192.168.1.3, Dst: 8.8.8.8
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 60
Identification: 0x4cc7 (19655)
000. .... = Flags: 0x0
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 128
Protocol: ICMP (1)
Header Checksum: 0x0000 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.1.3
Destination Address: 8.8.8.8
[Stream index: 22]



Capa (como la nombra Wireshark)	Dirección/identificador origen	Dirección/identificador destino	¿Qué campo indica qué protocolo viene "adentro"?
Ethernet II	f4:4e:b4:f8:d9:b1	8c:fd:de:60:76:a4	IPv4 (0x0800)
Internet Protocol Version 4	192.168.1.3	8.8.8.8	ICMP
Internet Control Message Protocol	No aplica	No aplica	8
Datos / payload	No aplica	No aplica	No aplica

a) La MAC destino del Echo Request enviado a 8.8.8.8, ¿es la MAC de 8.8.8.8? ¿De qué equipo es? Compárenla con la MAC destino del ping al gateway. ¿Qué conclusión sacan sobre el alcance de una dirección MAC frente al de una dirección IP?

No, no es la dirección MAC de 8.8.8.8 (el servidor de Google). Esa dirección física corresponde a nuestro router / gateway local (192.168.1.1).

Comparación: Al revisar notamos que la MAC destino del ping enviado a 8.8.8.8 y la MAC destino del ping enviado al gateway son exactamente la misma (8c:fd:de:60:76:a4).

Conclusión: Las direcciones MAC tienen un alcance estrictamente local (Capa 2 / Enlace); solo sirven para saltar de un dispositivo físico al siguiente dentro de la misma red hogareña o de la universidad. Por el contrario, las direcciones IP tienen alcance global / extremo a extremo (Capa 3 / Red); identifican al emisor y al receptor final de la comunicación sin importar cuántos países o routers haya en el medio. Nuestra PC no necesita saber la MAC de Google, solo necesita saber la MAC del router para "entregarle" el paquete y que este lo encamine hacia Internet.

b) Comparen un Echo Request con su Echo Reply (Wireshark los vincula en el campo [Response frame:...]). Hagan una lista de los campos que cambian y de los que se mantienen en Ethernet, IP e ICMP. ¿Por qué tiene sentido cada cambio? ¿Por qué el identificador y el número de secuencia se mantienen?

Capa / Protocolo	Campo	Echo Request (Paquete 423)	Echo Reply (Paquete 424)	¿Cambia o Se Mantiene?
Ethernet	MAC de Origen (Source)	00:e0:4c:68:1d:22	00:d0:56:a4:26:c4	**Cambia (Inversión)**
	MAC de Destino (Destination)	00:d0:56:a4:26:c4	00:e0:4c:68:1d:22	**Cambia (Inversión)**
IPv4	IP de Origen (Source Address)	192.168.1.12	192.168.1.1	**Cambia (Inversión)**
	IP de Destino (Destination Address)	192.168.1.1	192.168.1.12	**Cambia (Inversión)**
	Tiempo de Vida (TTL)	128	64	**Cambia**
	Identificación (Identification)	0x56a4 (22180)	0x1d22 (7458)	**Cambia**
	Checksum de la cabecera IP	0x4e0a	0x8b32	**Cambia**
ICMP	Tipo (Type)	8 (Echo Request)	0 (Echo Reply)	**Cambia**
	Código (Code)	0	0	**Se Mantiene**
	Identificador (Identifier)	0x0001	0x0001	**Se Mantiene**
	Número de Secuencia (Sequence Number)	0x002c (44)	0x002c (44)	**Se Mantiene**
	Checksum ICMP	0x4c4a	0x544a	**Cambia**
	Datos (Payload)	32 bytes (alfabeto)	32 bytes (alfabeto)	**Se Mantiene**

¿Por qué cambia cada campo?

• Inversión de Direcciones (MAC e IP): Tiene un sentido lógico bidireccional. El dispositivo que originalmente actuó como receptor (192.168.1.1 con MAC 00:d0:56:a4:26:c4) ahora toma el rol de emisor para responder, enviando la información de vuelta hacia el origen inicial (192.168.1.12 con MAC 00:e0:4c:68:1d:22). Las direcciones MAC se invierten porque representan el direccionamiento físico en el enlace local inmediato para este salto de red.
• Tiempo de Vida (TTL): Cambia de 128 en la solicitud a 64 en la respuesta debido a que cada sistema operativo inicializa su pila de red con un valor predeterminado diferente para los paquetes que genera (por ejemplo, Windows suele iniciar en 128 y Linux/Routers en 64). Además, disminuye según los saltos de enrutadores intermedios que atraviese.
• Identificación IP: La cabecera IPv4 requiere un número único de fragmentación para cada paquete enviado por un host. Al ser dos paquetes distintos e independientes generados por diferentes sistemas operativos en momentos de tiempo distintos, sus números de identificación son completamente diferentes (0x56a4 frente a 0x1d22).
• Tipo ICMP: Cambia conceptualmente de 8 a 0. Esto es necesario para que la máquina que originó el ping sepa que no está recibiendo otra solicitud, sino procesando la acción de respuesta definitiva a su consulta previa.
• Checksums (IP e ICMP): Ambos códigos de verificación matemática cambian obligatoriamente debido a que los datos contenidos dentro de las cabeceras sufrieron modificaciones (los campos de direcciones, TTL, tipos, etc., se alteraron), lo que invalida el cálculo anterior y exige generar uno nuevo para asegurar que el paquete no se corrompió en el tránsito.

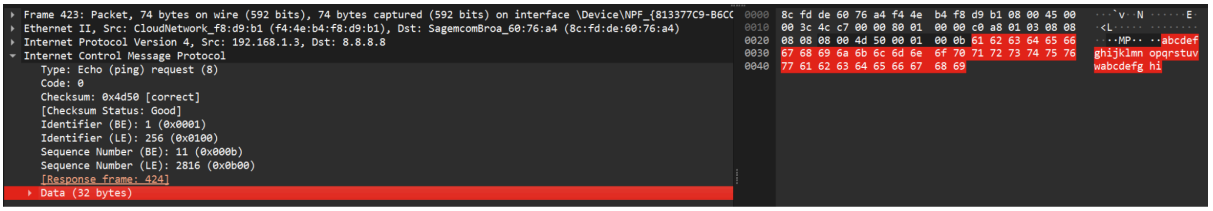
¿Por qué el Identificador, Número de Secuencia y Datos se mantienen idénticos?

• Code en 0: En los mensajes de Echo, el código siempre es 0 porque no existen subtipos o variantes operativas para esta función específica.
• Identifier y Sequence Number: Permanecer idénticos (0x0001 y 44 respectivamente) es la única vía que tiene el protocolo y el sistema operativo emisor para emparejar de forma unívoca la respuesta con su pregunta correspondiente. Al realizar ráfagas de pings simultáneos o consecutivos, la pila de red inspecciona exactamente estos dos campos para discriminar a qué solicitud en particular pertenece la respuesta que acaba de ingresar, logrando calcular de esta manera el tiempo exacto de ida y vuelta (Round-Trip Time o RTT).
• Payload (Datos): Los datos (en este caso la cadena de texto fija abcdefghijklmnopqrstuvwabcdefghi de 32 bytes) se devuelven intactos. Esto sirve para validar no solo que el canal tiene conectividad, sino que la integridad de la capa de aplicación se mantiene y los datos no sufren distorsiones o pérdidas de bits durante el transporte.

c) ¿Dónde está el payload de ping? ¿Cuántos bytes tiene y qué contiene? ¿Es igual en el Reply? Si en el grupo hay una computadora con Windows y otra con Linux, compárenlos: ¿qué les sugiere que sean distintos?

Se encuentra en la parte más profunda del paquete, al final de la estructura de ICMP (en Wireshark figura desplegado como "Data").

Tiene exactamente 32 bytes. Contiene una cadena de texto predefinida que el sistema operativo usa de relleno.



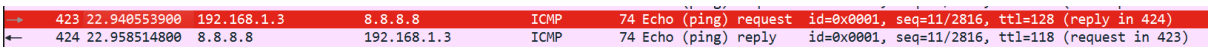
Sí, es idéntico. El servidor remoto está obligado a copiar el payload bit por bit de regreso para que la PC verifique que los datos no se corrompieron en el camino.

Si comparamos con una máquina Linux, veremos que Linux envía por defecto 56 bytes de payload (en lugar de 32) y su contenido suele ser una marca de tiempo estructurada (timestamp) seguida de caracteres numéricos/secuenciales. Esto sugiere que el protocolo ICMP no estandariza el contenido del mensaje, sino solo la estructura; cada sistema operativo tiene total libertad para rellenar el payload con el tamaño y los caracteres que sus desarrolladores hayan elegido.

d) ¿Qué valor de TTL tiene el Echo Request que ustedes enviaron? ¿Y el Reply que llegó de 8.8.8.8? ¿Por qué no son iguales? (Pista: investiguen qué hace un router con el TTL.)

Echo Request enviado: La máquina Windows lo envía con un TTL inicial de 128.

Echo Reply recibido: El paquete que volvió desde 8.8.8.8 llegó a la terminal con un TTL de 118.

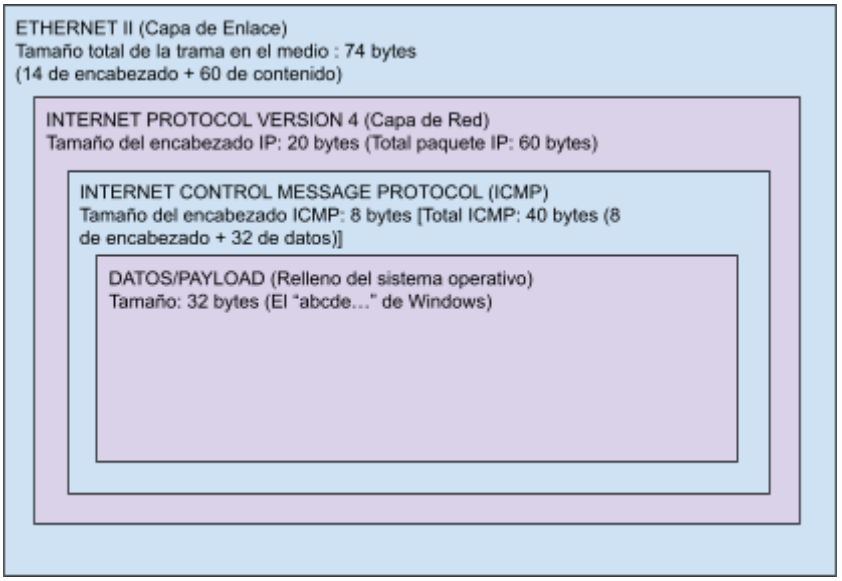


¿Por qué no son iguales?

Los valores no son iguales debido al mecanismo de control de bucles que implementa la capa de red del modelo OSI. Cada vez que un paquete IP atraviesa un router (o gateway) en Internet, este dispositivo procesa la cabecera e inevitablemente disminuye en 1 el valor del campo TTL antes de reenviarlo al siguiente salto. Si en algún punto el TTL llega a 0, el paquete se descarta y se envía un mensaje de error ICMP (Time Exceeded), evitando que los datos viajen infinitamente por la red en caso de existir un bucle de enrutamiento.

Conclusión técnica: Como no conocemos con certeza absoluta el valor de TTL inicial con el que el servidor 8.8.8.8 despachó el Echo Reply, no podemos determinar la cantidad exacta de saltos que realizó en su trayecto de vuelta. Lo único que la diferencia de valores nos demuestra de forma empírica es que el paquete sufrió un proceso de decremento secuencial por parte de los routers intermedios del camino desde que abandonó los servidores de Google hasta que ingresó a nuestra interfaz de red.

e) Dibujen la encapsulación del paquete que eligieron como "cajas dentro de cajas", indicando para cada caja qué tamaño en bytes tiene según Wireshark.



La caja exterior (Ethernet II) mide 74 bytes porque se compone de sus propios 14 bytes de encabezado local + los 60 bytes del paquete IP que lleva adentro (20 de IP + 8 de ICMP + 32 de datos = 60).

2. ARP: de una IP a una dirección MAC

a) ¿Qué problema resuelve ARP? ¿En qué capa lo ubicarían y por qué es discutible?

ARP (Address Resolution Protocol) permite obtener la dirección MAC correspondiente a una dirección IPv4 dentro de una red local. Aunque el equipo conozca la IP a la que debe enviar un paquete, necesita saber la dirección MAC del destino en el enlace local para construir la trama Ethernet. ARP establece esa asociación entre direcciones IP y MAC, denominada resolución de direcciones.

No es sencillo ubicar ARP en una sola capa del modelo OSI porque relaciona dos tipos de direccionamiento: emplea direcciones IPv4, propias de la capa de red, para obtener direcciones MAC, utilizadas en la capa de enlace. Por eso lo situamos más cerca de la capa de enlace, aunque su función vincula ambas capas.

b) ¿Qué es un ARP Request y un ARP Reply? ¿A quién se envía cada uno?

Un ARP Request es una solicitud enviada cuando un equipo conoce una dirección IPv4 de su red local, pero desconoce la dirección MAC asociada. Generalmente se transmite por broadcast para que todos los equipos del mismo dominio de broadcast reciban la consulta. El mensaje pregunta qué dispositivo tiene asignada la IP buscada.

Un ARP Reply es la respuesta del dispositivo que reconoce como propia la dirección IP consultada. En ella informa su MAC al equipo solicitante. A diferencia del Request inicial,

normalmente se envía por unicast, porque el equipo que responde ya conoce la MAC del solicitante gracias a los datos incluidos en la consulta.

c) ¿Qué es la caché ARP y por qué existe?

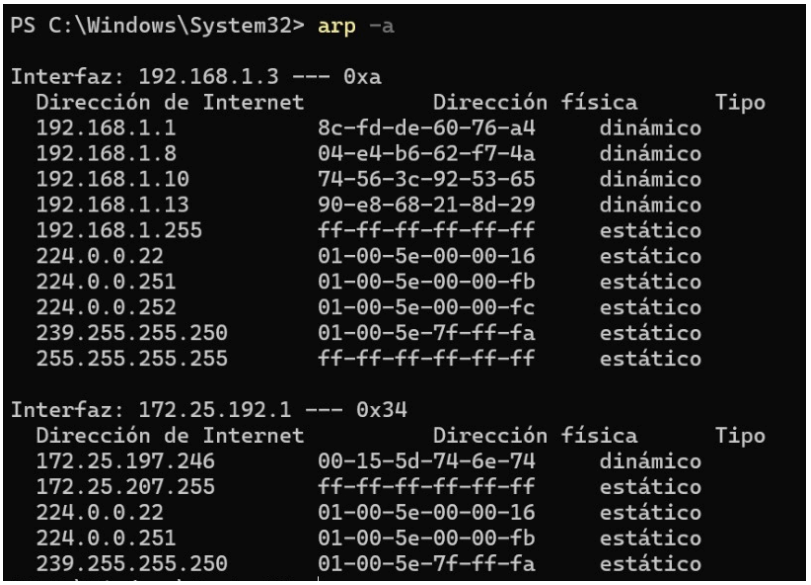
La caché ARP es una tabla temporal en la que el sistema operativo guarda asociaciones entre direcciones IPv4 y direcciones MAC que ha aprendido. Esto permite reutilizar una dirección MAC conocida sin repetir inmediatamente la resolución ARP.
Su objetivo es evitar consultas ARP innecesarias y agilizar la comunicación. Antes de generar una nueva solicitud, el equipo consulta la caché: si existe una asociación válida, la utiliza directamente; de lo contrario, debe volver a resolver la dirección. Así se reduce el tráfico broadcast de la red local.

d)"Tengo la IP de una máquina de mi red local. ¿Cómo sé a qué dirección MAC debo enviarle la trama?"

El equipo consulta primero su caché ARP. Si encuentra una asociación válida para la IP de destino, utiliza esa MAC para enviar la trama. De lo contrario, emite un ARP Request y espera la respuesta del equipo correspondiente. Cuando recibe el ARP Reply, guarda la asociación IP-MAC en la caché y puede transmitir la trama Ethernet.

e) Ver la caché ARP de su computadora y buscar la entrada del gateway. ¿La MAC asociada al gateway coincide con la MAC destino que vieron anteriormente?

Al ejecutar el comando arp -a, observamos que el gateway 192.168.1.1 está asociado con la MAC 8c:fd:de:60:76:a4. Esta dirección coincide con la MAC destino identificada anteriormente en Wireshark al enviar un Echo Request a 8.8.8.8. Esto confirma que, para comunicarse con una dirección IP externa a la red local, la computadora entrega inicialmente la trama Ethernet al gateway.



```
PS C:\Windows\System32> arp -a

Interfaz: 192.168.1.3 --- 0xa
  Dirección de Internet          Dirección física      Tipo
  192.168.1.1           8c-fd-de-60-76-a4     dinámico
  192.168.1.8           04-e4-b6-62-f7-4a     dinámico
  192.168.1.10          74-56-3c-92-53-65     dinámico
  192.168.1.13          90-e8-68-21-8d-29     dinámico
  192.168.1.255         ff-ff-ff-ff-ff-ff     estático
  224.0.0.22            01-00-5e-00-00-16     estático
  224.0.0.251           01-00-5e-00-00-fb     estático
  224.0.0.252           01-00-5e-00-00-fc     estático
  239.255.255.250       01-00-5e-7f-ff-fa     estático
  255.255.255.255       ff-ff-ff-ff-ff-ff     estático

Interfaz: 172.25.192.1 --- 0x34
  Dirección de Internet          Dirección física      Tipo
  172.25.197.246        00-15-5d-74-6e-74     dinámico
  172.25.207.255        ff-ff-ff-ff-ff-ff     estático
  224.0.0.22            01-00-5e-00-00-16     estático
  224.0.0.251           01-00-5e-00-00-fb     estático
  239.255.255.250       01-00-5e-7f-ff-fa     estático
```

f) Generar tráfico ARP y capturarlo.

Para estudiar ARP se realizaron capturas de tráfico en la interfaz de red. Primero se intentó resolver una dirección IPv4 local que no respondió y después se registró un intercambio con el gateway para analizar un ARP Request y su ARP Reply. Las capturas corresponden a dos configuraciones de red distintas: en el inciso e) se utilizó la red 192.168.1.0/24, con gateway 192.168.1.1; en este inciso se utilizó la red 192.168.0.0/24, con gateway 192.168.0.1. Por eso las direcciones IP y MAC registradas son diferentes y no deben compararse como si pertenecieran a una misma red. En ambos casos, ARP resuelve direcciones dentro del enlace local correspondiente. La siguiente tabla resume los campos observados.

Campo	**ARP Request**	**ARP Reply**
MAC destino (Ethernet)	ff:ff:ff:ff:ff:ff	d8:43:ae:97:2e:fd
MAC origen (Ethernet)	d8:43:ae:97:2e:fd	d8:07:b6:43:7e:36
Opcode	Request (1)	Reply (2)
Sender MAC address	d8:43:ae:97:2e:fd	d8:07:b6:43:7e:36
Sender IP address	192.168.0.102	192.168.0.1
Target MAC address	00:00:00:00:00:00	d8:43:ae:97:2e:fd
Target IP address	192.168.0.1	192.168.0.102

a) ¿Por qué el Request va a una dirección broadcast y el Reply no? ¿Qué valor tiene Target MAC address en el Request y por qué?

El ARP Request se envía habitualmente por broadcast porque el equipo conoce la IP buscada, pero todavía no sabe a qué dirección MAC corresponde. Por eso, en el encabezado Ethernet utiliza ff:ff:ff:ff:ff:ff como destino, de manera que la consulta alcance a los equipos del mismo dominio de broadcast. El ARP Reply normalmente se envía por unicast, ya que el dispositivo que responde conoce la MAC del solicitante a partir del Request. En la captura, el campo Target MAC Address del Request tiene el valor 00:00:00:00:00:00 porque esa dirección todavía se desconoce.

b) En el encabezado Ethernet de la trama ARP, ¿qué valor tiene el campo Type? ¿Hay un encabezado IP? ¿Qué les dice eso sobre dónde "vive" ARP?

En el encabezado Ethernet, el campo Type contiene 0x0806, correspondiente al protocolo ARP. No hay un encabezado IP entre Ethernet y ARP: el mensaje ARP se transporta directamente dentro de la trama Ethernet. Esto refuerza su relación con la capa de enlace, aunque emplea direcciones IPv4 para resolverlas en direcciones MAC.

c) Con la Opción A (IP inexistente): ¿cuántos ARP Request aparecieron? ¿Hubo Reply? ¿Apareció algún ICMP Echo Request en la captura? Expliquen por qué.

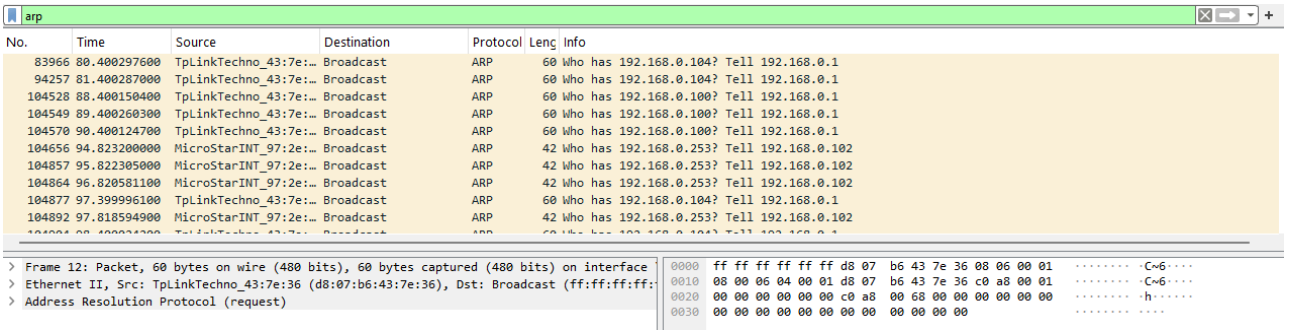
Al intentar hacer ping a 192.168.0.253, se observaron cuatro ARP Request y ningún ARP Reply en la captura filtrada por ARP. Por lo tanto, no se obtuvo una asociación IP-MAC para ese destino. Al no conocer una MAC a la cual dirigir la trama Ethernet, el equipo no puede transmitir el Echo Request hacia esa IP a través de la interfaz local. Para comprobar también la ausencia de tráfico ICMP es necesario aplicar el filtro arp || icmp, ya que el filtro arp, por sí solo, no muestra los mensajes ICMP.

d) Volvieron a hacer ping al mismo destino un minuto después: ¿apareció ARP de nuevo? Revisen la caché ARP. ¿Qué ventaja tiene la caché y qué problema podría causar si una entrada quedara vieja?

Al repetir el ping a 192.168.0.253 aproximadamente un minuto después, volvieron a aparecer solicitudes ARP con el mensaje «Who has 192.168.0.253? Tell 192.168.0.102». Al consultar la caché mediante arp -a, esa IP no figuraba entre las entradas. Como no se había recibido un ARP Reply, el equipo no pudo guardar una asociación válida y tuvo que volver a intentar la resolución.

La caché ARP permite reutilizar asociaciones IP-MAC conocidas, reduciendo la cantidad de solicitudes broadcast y el tiempo necesario para iniciar una comunicación. Sin embargo, una entrada desactualizada puede asociar una IP con una MAC incorrecta y provocar fallas o el envío de tramas al equipo equivocado hasta que la entrada se actualice o elimine.

Captura del segundo intento de resolución ARP:



Comprobación de la caché ARP mediante arp -a:

```
PS C:\WINDOWS\system32> arp -a

Interfaz: 192.168.0.102 --- 0x3
  Dirección de Internet          Dirección física      Tipo
  192.168.0.1           d8-07-b6-43-7e-36     dinámico
  192.168.0.255         ff-ff-ff-ff-ff-ff     estático
  224.0.0.2             01-00-5e-00-00-02     estático
  224.0.0.22            01-00-5e-00-00-16     estático
  224.0.0.251           01-00-5e-00-00-fb     estático
  224.0.0.252           01-00-5e-00-00-fc     estático
  224.168.100.1         01-00-5e-28-64-01     estático
  239.255.255.250       01-00-5e-7f-ff-fa     estático
  255.255.255.255       ff-ff-ff-ff-ff-ff     estático

Interfaz: 172.17.112.1 --- 0x25
  Dirección de Internet          Dirección física      Tipo
  172.17.114.72         00-15-5d-62-48-93     dinámico
  172.17.127.255        ff-ff-ff-ff-ff-ff     estático
  224.0.0.2             01-00-5e-00-00-02     estático
  224.0.0.22            01-00-5e-00-00-16     estático
  224.0.0.251           01-00-5e-00-00-fb     estático
  239.255.255.250       01-00-5e-7f-ff-fa     estático
```

3) TCP y UDP "a mano" con ncat

a) ¿Qué significa "establecer una conexión"? ¿Dónde "existe" una conexión TCP: en los cables, en los routers o en los extremos?

TCP requiere que las dos aplicaciones que van a comunicarse acuerden explícitamente participar en la transferencia antes de enviar datos de usuario.Se le dice “establecer una conexión” a este acuerdo entre los programas de aplicación y los módulos de protocolo TCP de ambos extremos.
Para hacerlo, TCP utiliza el three-way handshake: el cliente envía un segmento **SYN**, el servidor responde con **SYN+ACK** y el cliente finaliza con un **ACK**. Este intercambio permite sincronizar los números de secuencia y establecer el estado necesario para la comunicación.
La conexión TCP existe como un estado mantenido por los dos hosts extremos de la comunicación. Los routers intermedios no mantienen la conexión TCP, en cambio: reciben y reenvían datagramas IP de manera independiente.

b) ¿Qué es un puerto? ¿Qué identifica el par (IP, puerto)?

Un puerto es un número de 16 bits utilizado por TCP o UDP para identificar un punto de comunicación dentro de un host y permitir que los datos sean entregados a la aplicación correspondiente. La dirección IP permite identificar el host, y el puerto permite distinguir el servicio o proceso dentro de ese host.
La combinación *(IP, puerto)* identifica un endpoint o punto final de comunicación. Por ejemplo, 127.0.0.1:12000 identifica el puerto 12000 de la máquina local. En TCP, una conexión queda identificada por los dos endpoints: por la combinación de IP y puerto de origen y de destino.

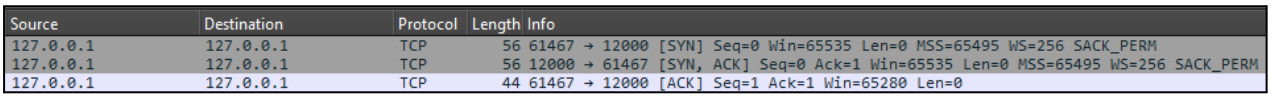
c) ¿Qué significa que un proceso esté "escuchando" en un puerto?

Un proceso que esté "escuchando" en un puerto significa que ha ejecutado una apertura pasiva (passive open), notificando al sistema operativo que se encuentra listo para recibir solicitudes entrantes en un puerto local específico (pasando al estado *LISTEN*). Mediante la función ***listen()***, el kernel asigna una cola de peticiones (backlog) para absorber ráfagas de clientes. Mientras el servidor está en este modo, la pila TCP completa de forma transparente el *three-way handshake* con cada emisor y deposita las conexiones ya establecidas en esa cola.
Al invocar ***accept()***, extrae la conexión de la cola y el sistema operativo genera un nuevo socket (connection socket) dedicado exclusivamente a ese cliente (identificado por su 4-tupla completa), permitiendo que el socket de acogida original continúe escuchando sin interrupción. Si llega un segmento **SYN** a un puerto donde ningún proceso escucha, TCP rechaza la solicitud de inmediato enviando un Reset (RST), o un mensaje ICMP Port Unreachable en UDP. Si el puerto sí escucha pero el backlog se encuentra saturado, el SO descarta el **SYN** para forzar un reintento posterior.

Para responder las preguntas referidas a ambos experimentos se siguieron las consignas al realizarlos desde dos terminales en la misma computadora, una teniendo el rol de servidor y la otra de cliente. Las capturas de dichas conexiones se encuentran junto al informe.

a) ¿Qué pasó en la red cuando ejecutaron el comando del cliente, antes de escribir el primer mensaje? Compárenlo con TCP.

En el protocolo TCP, al ejecutar el comando del cliente se produjo primero el establecimiento de la conexión antes de enviar los datos. En la captura se observa el three-way handshake, compuesto por un segmento **SYN** que fue enviado por el cliente, un **SYN + ACK** enviado por el servidor de regreso y un **ACK** enviado nuevamente por el cliente. Al completarse este intercambio, quedó la conexión establecida y se puede comenzar a enviarse los datos.

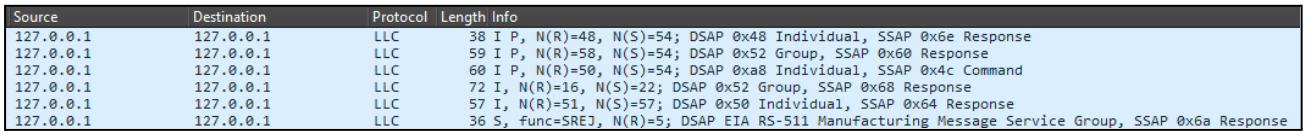


Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	TCP	56	61467 → 12000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
127.0.0.1	127.0.0.1	TCP	56	12000 → 61467 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
127.0.0.1	127.0.0.1	TCP	44	61467 → 12000 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Pero con UDP no se realiza un procedimiento equivalente antes de transmitir información. Por lo que no se vió el envío de segmentos propios del three-way handshake para establecer una conexión previamente.

b) ¿Cuántos datagramas generó cada mensaje? ¿Hay algo parecido a un ACK?

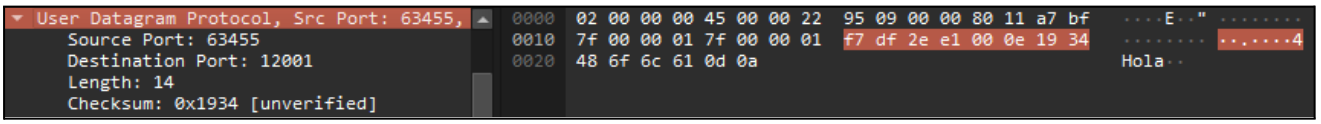
En la captura de UDP se puede observar que cada mensaje enviado genera un datagrama UDP diferente. La cantidad exacta de datagramas se determinó observando los paquetes generados en Wireshark que para un intercambio de 6 mensajes generó un datagrama para cada uno de estos. Como este protocolo no posee un mecanismo de confirmación de recepción equivalente al ACK de TCP, no se observa un ACK generado automáticamente por el protocolo UDP después de cada mensaje.



Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=48, N(S)=54; DSAP 0x48 Individual, SSAP 0x6e Response
127.0.0.1	127.0.0.1	LLC	59	I P, N(R)=58, N(S)=54; DSAP 0x52 Group, SSAP 0x60 Response
127.0.0.1	127.0.0.1	LLC	60	I P, N(R)=50, N(S)=54; DSAP 0xa8 Individual, SSAP 0x4c Command
127.0.0.1	127.0.0.1	LLC	72	I, N(R)=16, N(S)=22; DSAP 0x52 Group, SSAP 0x68 Response
127.0.0.1	127.0.0.1	LLC	57	I, N(R)=51, N(S)=57; DSAP 0x50 Individual, SSAP 0x64 Response
127.0.0.1	127.0.0.1	LLC	36	S, func=SREJ, N(R)=5; DSAP EIA RS-511 Manufacturing Message Service Group, SSAP 0x6a Response

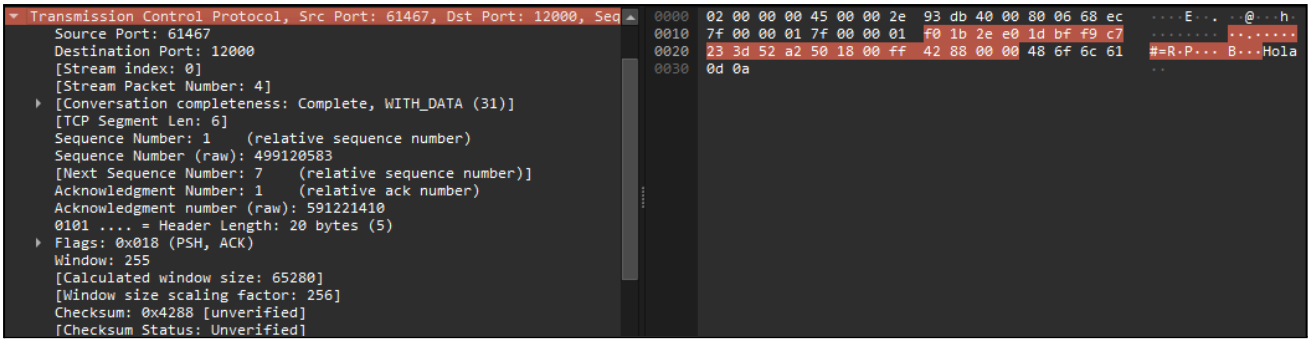
c) Comparen el encabezado UDP con el encabezado TCP de un segmento con datos: ¿qué campos tiene cada uno? ¿Cuántos bytes ocupa cada encabezado?

El encabezado UDP está compuesto por cuatro campos y ocupa un total fijo de 8 bytes (2 para cada campo):



- *Source port*: puerto de origen.
- *Destination port*: puerto de destino.
- *Length*: longitud total del datagrama UDP.
- *Checksum*: mecanismo de detección de errores.

Pero la cantidad de bytes que ocupa un encabezado TCP varía dependiendo de las opciones que incluya, este tiene un tamaño mínimo de 20 bytes.



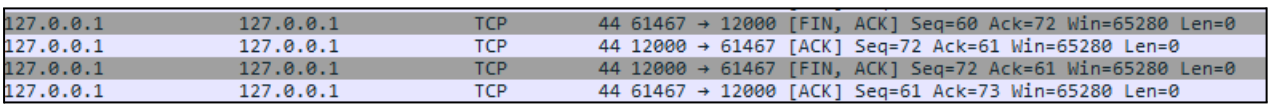
Al contener una cantidad mayor de información, ya que se debe mantener el estado de la conexión y proporcionar mecanismos de comunicación confiable. Por esto, TCP posee un encabezado más grande que UDP al transportar información adicional relacionada con el establecimiento, mantenimiento, control y cierre de la conexión, sumado a los mecanismos de secuenciamiento y reconocimiento de los datos.

Los principales campos de TCP son los siguientes:

- *Source port*: puerto de origen.
- *Destination port*: puerto de destino.
- *Sequence Number:* número de secuencia asociado a los datos.
- *Acknowledgment Number*: número utilizado para reconocer datos recibidos.
- *Header Length*: longitud del encabezado TCP.
- *Flags*: permiten controlar distintos estados y operaciones de la conexión, como **SYN**, **ACK**, **FIN** y **RST**.
- *Window*: utilizado para el control de flujo con la cantidad de datos que el receptor puede aceptar antes de necesitar una confirmación.
- *Checksum*: detección de errores.
- *Urgent pointer*: utilizado para información urgente.
- *TCP Segment Len*: tamaño en bytes de los datos de la aplicación que se transportan en el segmento TCP.

d) ¿Qué pasó en la red al cerrar el cliente con Ctrl+C? ¿Y en TCP?

En TCP, al existir una conexión establecida, su cierre se realiza al conseguir completar un intercambio de segmentos de control. Para una finalización se puede observar segmentos que contienen la bandera **FIN**, junto con los correspondientes **ACK**, para indicar que un extremo terminó de enviar datos y permitir el cierre de la conexión.



Por otro lado, al cerrar el cliente UDP con *Ctrl+C*, no se realiza un procedimiento de cierre equivalente al realizado en el protocolo TCP, esto se debe a que UDP no establece previamente una conexión, ergo, la finalización del programa no requiere un intercambio de segmentos de control para cerrar una conexión.

En UDP la finalización del proceso no implica un protocolo de cierre de conexión, mientras que en TCP el cierre forma parte del funcionamiento del protocolo y se puede observar en los segmentos de control correspondientes.

e) **Para enviar la misma frase, ¿cuántos paquetes necesitaron en total con TCP y cuántos con UDP? ¿Qué "compran" con los paquetes extra de TCP?**

En la captura se observaron 9 segmentos TCP asociados a la comunicación: tres correspondientes al establecimiento de la conexión, dos relacionados con el envío y reconocimiento de los datos y cuatro correspondientes al cierre de la conexión. Mientras que para el caso de UDP, la frase fue enviada mediante un único datagrama UDP, ya que este protocolo no necesita establecer ni cerrar una conexión y tampoco utiliza un mecanismo de reconocimiento equivalente al de TCP.

Estos paquetes adicionales permiten obtener características como una comunicación orientada a conexión, entrega confiable, ordenamiento de los datos, control de flujo y mecanismos para detectar y recuperar pérdidas que UDP no proporciona. Estos paquetes adicionales garantizan cierta confiabilidad y control sobre la comunicación, por otro lado, UDP prioriza una menor sobrecarga y el envío directo de datagramas al no establecer previamente una conexión ni garantizar por sí mismo la entrega, el orden o la retransmisión de los datos.

f) **¿Y si nadie escucha? Con Wireshark capturando en loopback y el filtro tcp.port == 12000 || udp.port == 12001 || icmp, sin servidores corriendo:**

```
ncat -v 127.0.0.1 12000          # TCP a un puerto cerrado
ncat -v -u 127.0.0.1 12001       # UDP a un puerto cerrado, escribir y enter
```

En TCP el cliente envía un segmento SYN para iniciar el three-way handshake. Al no haber una terminal de servidor escuchando en ese puerto, el intento de conexión es rechazado mediante un segmento TCP con la bandera RST+ACK. Por lo que el intento de establecimiento de la conexión es seguido por la respuesta de rechazo del sistema operativo.

Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	TCP	56	12571 → 12000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
127.0.0.1	127.0.0.1	TCP	44	12000 → 12571 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
127.0.0.1	127.0.0.1	TCP	56	[TCP Port numbers reused] 12571 → 12000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM

Mientras que en UDP el cliente envía el datagrama del protocolo aunque no haya ningún proceso asociado al puerto de destino. Cuando el sistema operativo determina que el puerto no está disponible, genera un mensaje ICMP Destination Unreachable para indicar que no existe una aplicación disponible para recibir el datagrama.

Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=48, N(S)=54; DSAP 0x48 Individual, SSAP 0x6e Response
127.0.0.1	127.0.0.1	ICMP	66	Destination unreachable (Port unreachable)

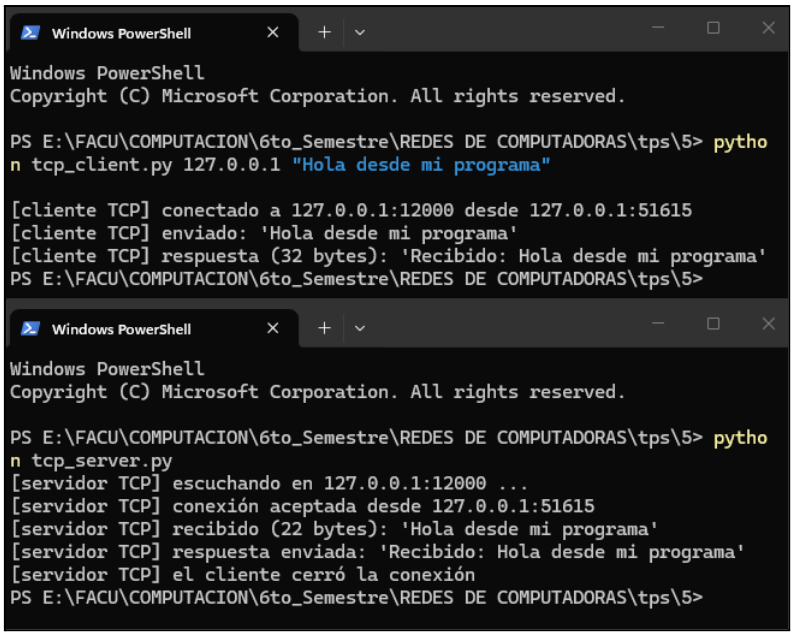
Para TCP, se detecta el puerto cerrado durante el intento de establecimiento de la conexión y responde mediante un segmento con RST. UDP no establece conexión y el sistema operativo informa que el puerto no está disponible mediante un mensaje ICMP.

4) “Servidor TCP mínimo“:

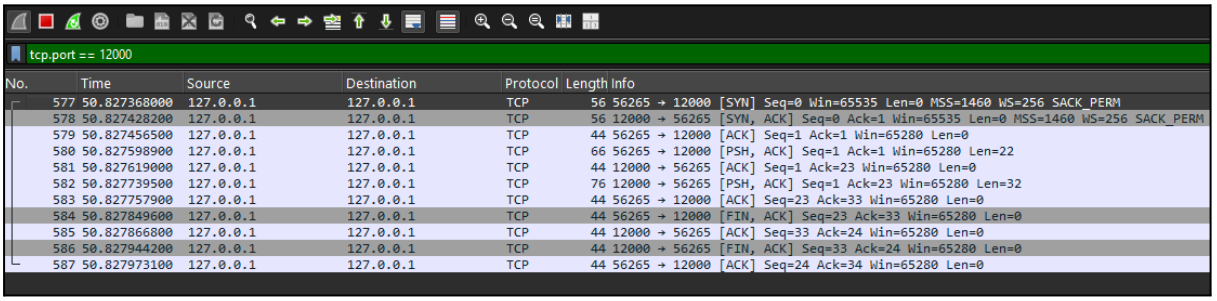
c) Capturando loopback en Wireshark utilizando el filtro “tcp.port == 12000” y, junto a dos terminales ejecutando los comandos:

- python tcp_client.py 127.0.0.1 "Hola desde mi programa"
- python tcp_server.py

Obtenemos, en las consolas, las siguientes salidas:



Mientras que en wireshark obtendremos la siguiente captura:



d) Con la captura, completar una tabla como la siguiente. Para cada llamada, indicar si genera tráfico y, si lo genera, qué segmentos (número de paquete en Wireshark y flags):

Llamada	¿Dónde se ejecuta?	¿Genera tráfico?	Segmentos que observan
socket()	**servidor y cliente**	No	Ninguno

bind()	**servidor**	No	Ninguno
listen()	**servidor**	No	Ninguno
connect()	**cliente**	Sí (el OS arranca el handshake en el)	Paquetes 577 [SYN], 578 [SYN, ACK] y 579 [ACK].
accept()	**servidor**	No	Ninguno (acepta la conexión ya establecida en los paquetes 577 al 579).
sendall()	**ambos**	Sí (el OS transmite los datos)	Paquetes 580 [PSH, ACK] y 582 [PSH, ACK] (el sistema operativo transmite los datos del cliente y del servidor).
recv()	**ambos**	No	Se observan los paquetes 581 [ACK] y 583 [ACK], pero son generados como parte del funcionamiento de TCP.
close()	**ambos**	Si	Paquete 584 [FIN, ACK] (inicio del cierre), 585 [ACK], 586 [FIN, ACK] y 587 [ACK] (cierre final).

- **socket():** Como se menciona en el pdf de la consigna, socket es una operación local, es decir que no viaja por la red, o sea, no generará tráfico.
- **bind():** Sirve para asociar el socket a una IP y puerto, ambos locales (local, sin tráfico).
- listen(): Funciona para pedirle al OS que acepte conexiones en dicho puerto, y es local.
- **connect():** Es una llamada bloqueante del lado del cliente que solicita al Sistema Operativo iniciar la conexión (enviando el paquete SYN). El programa se detiene en esa línea hasta que el three-way handshake se completa exitosamente (o lanza un timeout si no recibe respuesta).
- **accept():** Es una llamada bloqueante del lado del servidor: el programa se detiene en esa línea esperando a que un cliente inicie y complete el three-way handshake, momento en el cual se desbloquea y devuelve un socket nuevo dedicado exclusivamente a esa conexión.
- **sendall():** Recibe un buffer de bytes y lo envía al OS, el cual lo procesa para transmitirlo por la red. A nivel de red, se observan segmentos con la bandera PSH (Push) que transportan los datos reales del mensaje y solicitan al receptor que los procese de inmediato sin acumularlos en buffer.
- **recv():** Se encarga de leer y extraer los bytes que estén almacenados en el buffer del socket. Si el buffer está vacío, se queda esperando de forma bloqueante hasta que lleguen los datos. A nivel de red, la llegada de estos datos hace que el sistema operativo responda automáticamente con un segmento ACK de confirmación.
- **close():** Es una solicitud que ocurre tanto en el cliente como en el servidor donde se liberan los recursos locales utilizados en el proceso y se hace la petición al sistema operativo para iniciar el proceso de cierre de la conexión. A nivel de red, la pila TCP de cada extremo envía un segmento con la bandera FIN (Finalize) para indicar que ya no transmitirá más datos, el cual es respondido por un segmento ACK de confirmación desde el otro extremo."

REFERENCIAS

1. Stallings, W. (2004). *Comunicaciones y redes de computadores* (7.ª ed.). Pearson Educación.
2. Comer, D. Internetworking with TCP/IP, Vol. 1 (6.ª ed. o superior). Pearson.
3. Kurose, J. y Ross, K. Redes de computadores: un enfoque descendente (7.ª ed. o superior). Addison Wesley.